Run Time Calculation

CS422 Assignment 3 - Group 6

Aman Dixit
230112

Aditya Gautam
220064

Abhijeet Agarwal
210025

Saaumitra Raaj
220928

Methodology and Formulas

As per the assignment guidelines, the total run time (in CPU clock cycles) is calculated by assigning specific cycle weights to different instruction types. We extract the total instruction count and memory access count using the Intel Pin dynamic instrumentation tool.

The formulas used for these calculations are:

$$\text{ALU Instructions} = \text{Total Instructions} - \text{Memory Accesses} \tag{1}$$

$$\text{Total Run Time (Cycles)} = (\text{Memory Accesses} \times 5) + (\text{ALU Instructions} \times 1) \tag{2}$$

$$\text{Average CPI} = \frac{\text{Total Run Time (Cycles)}}{\text{Total Instructions}} \tag{3}$$

Part 1: Benchmarks Without Compiler Optimization (Default)

1.1 Custom Secure Implementation (Default)

- **Total Instructions:** $548,335$
- **Memory Accesses:** $195,896$

Step 1: Calculate ALU / Logic Instructions

$$\begin{aligned}\text{ALU Instructions} &= 548,335 - 195,896 \\ &= \mathbf{352{,}439}\end{aligned}$$

Step 2: Calculate Total Run Time (Cycles)

$$\begin{aligned}\text{Total Cycles} &= (195,896 \times 5) + (352,439 \times 1) \\ &= 979,480 + 352,439 \\ &= \mathbf{1{,}331{,}919\ Cycles}\end{aligned}$$

Step 3: Calculate Average CPI

$$\text{Average CPI} = \frac{1,331,919}{548,335} \approx \mathbf{2.43}$$

1.2 OpenSSL Baseline (Default)

- **Total Instructions:** $502,362$
- **Memory Accesses:** $168,363$

Step 1: Calculate ALU / Logic Instructions

$$\begin{aligned}\text{ALU Instructions} &= 502,362 - 168,363 \\ &= \mathbf{333{,}999}\end{aligned}$$

Step 2: Calculate Total Run Time (Cycles)

$$\begin{aligned}\text{Total Cycles} &= (168,363 \times 5) + (333,999 \times 1) \\ &= 841,815 + 333,999 \\ &= \mathbf{1{,}175{,}814\ Cycles}\end{aligned}$$

Step 3: Calculate Average CPI

$$\text{Average CPI} = \frac{1,175,814}{502,362} \approx \mathbf{2.34}$$

1.3 Relative Run Time (Default)

$$\text{Relative Run Time} = \frac{1,331,919}{1,175,814} \approx \mathbf{1.132x}\ (13.2\%\ \text{overhead})$$

Part 2: Benchmarks With Compiler Optimization (-O4)

2.1 Custom Secure Implementation (-O4)

- **Total Instructions:** $521,382$
- **Memory Accesses:** $168,589$

Step 1: Calculate ALU / Logic Instructions

$$\begin{aligned}\text{ALU Instructions} &= 521,382 - 168,589 \\ &= \mathbf{352{,}793}\end{aligned}$$

Step 2: Calculate Total Run Time (Cycles)

$$\begin{aligned}\text{Total Cycles} &= (168,589 \times 5) + (352,793 \times 1) \\ &= 842,945 + 352,793 \\ &= \mathbf{1{,}195{,}738\ Cycles}\end{aligned}$$

Step 3: Calculate Average CPI

$$\text{Average CPI} = \frac{1,195,738}{521,382} \approx \mathbf{2.29}$$

2.2 OpenSSL Baseline (-O4)

- **Total Instructions:** $502,170$
- **Memory Accesses:** $168,222$

Step 1: Calculate ALU / Logic Instructions

$$\begin{aligned}\text{ALU Instructions} &= 502,170 - 168,222 \\ &= \mathbf{333{,}948}\end{aligned}$$

Step 2: Calculate Total Run Time (Cycles)

$$\begin{aligned}\text{Total Cycles} &= (168,222 \times 5) + (333,948 \times 1) \\ &= 841,110 + 333,948 \\ &= \mathbf{1{,}175{,}058\ Cycles}\end{aligned}$$

Step 3: Calculate Average CPI

$$\text{Average CPI} = \frac{1,175,058}{502,170} \approx \mathbf{2.34}$$

2.3 Final Relative Run Time Comparison (`-O4`)

$$\begin{aligned}\text{Relative Run Time} &= \frac{\text{Custom Total Cycles}}{\text{OpenSSL Total Cycles}} \\ &= \frac{1,195,738}{1,175,058} \\ &\approx \mathbf{1.0176x}\end{aligned}$$

Conclusion: our custom constant-time implementation runs with a relative run time of **1.0176x** compared to the highly optimized OpenSSL baseline,